

FloatChat - Development Report & Interview Q&A

The build process, the questions it invites, terms answered, and sample code

Nirmalya Mandal - SAP Labs Interview Prep

Reconstructed from the project files and pipeline

Contents

How to use this report	3
Part 1 - The development story	4
Phase 1 - Getting the data (the ETL pipeline)	4
Phase 2 - The backend and natural-language-to-SQL	4
Phase 3 - RAG and vector search	4
Phase 4 - The frontend and visualisation	4
Phase 5 - Deployment	4
Part 2 - Questions and detailed answers	5
“What is NetCDF, and why an ETL step?”	5
“Why FastAPI for the backend?”	5
“How does the natural-language-to-SQL actually work?”	5
“How do you stop the model from generating dangerous SQL?”	5
“What is RAG, and why use it?”	5
“What is a vector database / an embedding?”	5
“Why Mistral 7B instead of GPT-4?”	6
“Why PostgreSQL for scientific data?”	6
Part 3 - The bait-terms dictionary	7
Part 4 - Sample code snippets (pen and paper)	9
The hooks to drop on purpose	11

How to use this report

FloatChat is your AI project, and it maps directly onto what SAP does with Joule: natural language in, real answers from real data out. That single sentence is your best bridge to the company, so this report prepares you to defend every piece of it.

Structure: **Part 1** the build story, **Part 2** questions with detailed answers, **Part 3** the bait-terms dictionary, **Part 4** sample code for the pen-and-paper round. Say a term, pause, let them ask - and only drop what you can hold.

What FloatChat is, in one breath: an AI conversational analytics platform that lets a researcher ask a plain-English question about 23 years of Argo ocean-float data (roughly 79,934 profiles from 451 floats) and get back real answers as maps, depth charts, and a 3D globe - by turning the question into a validated SQL query with a language model.

Honesty note: this was a team/hackathon-style build around the INCOIS Argo problem. Speak to the parts you built and understand; don't claim the whole thing single-handed if asked directly. The tech below is real and defensible.

Part 1 - The development story

Phase 1 - Getting the data (the ETL pipeline)

- The raw data is **Argo** ocean-float profiles in **NetCDF** (.nc) scientific files, plus some legacy Microsoft **Access** (.accdb) tables.
- Wrote a **downloader** (script.py) that pulls Argo profiles from the **IFREMER** data server using its global index file - **streaming** the files, skipping ones already downloaded, and logging errors.
- Converted the Access .accdb tables to **CSV** with pyodbc + pandas.
- Cleaned and standardised the NetCDF data in notebooks using **xarray** and **pandas**.
- Loaded the cleaned CSVs into **PostgreSQL** (csv_to_postgreSQL).
- The whole point: do the heavy processing **once**, up front, so every user query later hits fast, indexed database tables instead of slow raw files.

Phase 2 - The backend and natural-language-to-SQL

- Built an **async FastAPI** backend to handle queries and orchestrate the AI.
- Used **LangChain** to wire the steps together, with **Mistral 7B** (an open LLM, run via **HuggingFace**) doing the generation.
- **Schema-aware prompting**: the model is told the exact table structure, so it generates SQL that fits the real schema.
- **Query validation before execution**: generated SQL is checked (read-only, no destructive statements) before it ever runs.

Phase 3 - RAG and vector search

- Added **Retrieval-Augmented Generation**: before answering, retrieve relevant context so the model is grounded.
- Stored **embeddings** in a **vector database** (Upstash Vector / Supabase) and retrieved similar past queries by **similarity** to guide the model.
- **Supabase** provided the PostgreSQL database and vector storage together.

Phase 4 - The frontend and visualisation

- Built the frontend in **Next.js** + TypeScript + Tailwind.
- Rendered results as **2D maps with Leaflet.js**, a **3D globe with Globe.gl**, and depth/profile charts.
- Shaped the visualisation data on the server to keep the browser light.

Phase 5 - Deployment

- Deployed the frontend on **Vercel**; the backend ran in **Docker** behind an **Nginx** reverse proxy.

Part 2 - Questions and detailed answers

“What is NetCDF, and why an ETL step?”

NetCDF is a scientific file format for multi-dimensional array data - ocean measurements indexed by float, depth, and time. It is great for science but slow to query directly and awkward for a web app. So I built an **ETL** pipeline - Extract from IFREMER, Transform and clean with xarray/pandas, Load into PostgreSQL - to do that expensive work once. After that, every user query hits fast indexed SQL tables, not raw files.

“Why FastAPI for the backend?”

The heavy lifting here is AI and data work, which lives in the Python ecosystem (LangChain, model libraries, xarray). FastAPI is a modern, lightweight, **async** Python framework built for exactly this - API endpoints that call models and process data - with automatic request validation. Async matters because an LLM call spends most of its time waiting on the model, and async lets the server handle other requests during that wait.

“How does the natural-language-to-SQL actually work?”

The user’s English question goes to the LLM along with the database schema (this is **schema-aware prompting**), and the model returns a SQL query. Before running it, I **validate** it - it must be read-only, no destructive statements - so a bad or unsafe query never touches the database. Then it runs against PostgreSQL and the results are visualised. The reliability comes from the engineering around the model, not just the model.

“How do you stop the model from generating dangerous SQL?”

Three layers. The model only ever needs to read, so I validate that the generated query is a **SELECT** and reject anything with destructive keywords (**DROP**, **DELETE**, **UPDATE**, **INSERT**). The database user the app connects as has read-only permissions as a backstop. And schema-aware prompting keeps the model on the real tables. Defence in depth - never trust the model’s output blindly.

“What is RAG, and why use it?”

Retrieval-Augmented Generation. Before the model answers, I retrieve relevant context - here, similar earlier queries and schema information - and feed it into the prompt, so the answer is grounded in real information instead of the model guessing. It reduces hallucination and makes the SQL more reliable.

“What is a vector database / an embedding?”

An embedding turns text into a list of numbers that captures its meaning, so two questions that mean similar things get similar vectors. A vector database stores these and lets me search by **meaning** rather than exact keywords - “find the past queries most similar to this one” - usually by cosine similarity between vectors. I used Upstash Vector / Supabase for that.

“Why Mistral 7B instead of GPT-4?”

A 7-billion-parameter open model gives predictable cost and latency, which matters when every chat message triggers an inference. A frontier model would generate slightly better SQL but at higher, less predictable cost. I made the smaller model reliable with schema-aware prompting, validation, and RAG - engineering over raw size.

“Why PostgreSQL for scientific data?”

Once the data is cleaned it is highly structured - measurements with floats, depths, timestamps, coordinates - and queries are relational (filter by region, by year, by depth). PostgreSQL gives fast indexed queries and, with PostGIS-style handling, good geospatial support. It is the right home for structured, query-heavy data.

Part 3 - The bait-terms dictionary

NetCDF

A scientific file format for multi-dimensional array data (ocean profiles). *Say it:* “The raw Argo data is NetCDF - great for science, so I ETL’d it into PostgreSQL for fast queries.”

ETL

Extract, Transform, Load - pull data, reshape it, load it into your database. *Say it:* “I built an ETL pipeline: download from IFREMER, clean with xarray, load into Postgres.”

xarray

A Python library for working with labelled multi-dimensional (NetCDF) data. *Say it:* “I cleaned the NetCDF with xarray and pandas before loading it.”

FastAPI (async)

A modern, fast, async Python API framework. *Say it:* “FastAPI, async because LLM calls are mostly waiting - async keeps the server busy.”

LangChain

A framework for wiring LLM app steps together. *Say it:* “LangChain orchestrates the pipeline - question, schema context, model, validate, run.”

Mistral 7B / HuggingFace

An open 7-billion-parameter LLM, run via HuggingFace. *Say it:* “Mistral 7B for predictable cost and latency, made reliable by good prompting.”

NL-to-SQL / schema-aware prompting

Turning a plain-English question into SQL, guided by the real table schema. *Say it:* “I feed the model the schema so its SQL fits the real tables.”

Query validation

Checking generated SQL is safe (read-only) before running it. *Say it:* “Every generated query is validated - SELECT-only - before it touches the DB.”

RAG

Retrieval-Augmented Generation - retrieve real context, then generate grounded. *Say it:* “RAG grounds the model in similar past queries and the schema.”

Embedding / vector database / cosine similarity

Text turned into meaning-vectors, stored for similarity search. *Say it:* “Embeddings let me find semantically similar queries by cosine similarity in a vector DB.”

Supabase / Upstash Vector

Managed PostgreSQL (Supabase) and vector storage (Upstash) . *Say it:* “Supabase for Postgres, Upstash for the vectors.”

Leaflet.js / Globe.gl

2D map library and 3D globe library. *Say it:* “Results render as Leaflet 2D maps and a Globe.gl 3D globe.”

Streaming download

Downloading a file in chunks instead of all into memory. *Say it:* “The downloader streams each NetCDF and skips ones already fetched.”

Part 4 - Sample code snippets (pen and paper)

Short and defensible. Narrate the logic as you write.

The Argo downloader (streaming, skip-existing)

```
import requests, os

def download(url, path):
    if os.path.exists(path):          # don't re-download
        return
    with requests.get(url, stream=True) as r:
        r.raise_for_status()
        with open(path, "wb") as f:
            for chunk in r.iter_content(1024 * 256): # stream in chunks
                f.write(chunk)
```

Say aloud: “I stream each file in chunks so a big NetCDF never loads fully into memory, and skip anything already downloaded.”

Natural-language to SQL (LangChain-style)

```
PROMPT = """You are a SQL assistant. Schema:
{schema}
Write ONE read-only SQL SELECT for: {question}"""

def to_sql(question, schema, llm):
    prompt = PROMPT.format(schema=schema, question=question)
    return llm.invoke(prompt).strip()
```

Say aloud: “The schema goes into the prompt - schema-aware prompting - so the model’s SQL matches the real tables.”

Validate the SQL before running it

```
BANNED = ("drop", "delete", "update", "insert", "alter", ";--")

def is_safe(sql):
    low = sql.lower().strip()
    return low.startswith("select") and not any(b in low for b in BANNED)
```

Say aloud: “It must be a SELECT and contain no destructive keywords - I never run the model’s output blindly.”

RAG retrieval (top-k similar queries)

```
def retrieve_context(question, embed, vector_db, k=3):
    q_vec = embed(question)          # question -> embedding
    hits = vector_db.query(q_vec, top_k=k) # nearest by similarity
    return "\n".join(h.text for h in hits)
```

Say aloud: “I embed the question and pull the k most similar past queries as grounding context.”

FastAPI async endpoint

```
from fastapi import FastAPI
app = FastAPI()

@app.post("/ask")
async def ask(question: str):
    context = retrieve_context(question, embed, vdb)
    sql = to_sql(question, SCHEMA + context, llm)
    if not is_safe(sql):
        return {"error": "unsafe query"}
    rows = await db.fetch(sql)
    return {"sql": sql, "rows": rows}
```

Say aloud: “Async because the model call is mostly waiting; retrieve, generate, validate, run.”

Cleaning NetCDF with xarray

```
import xarray as xr

ds = xr.open_dataset("profile.nc")          # open NetCDF
df = ds[["TEMP", "PSAL", "PRES"]].to_dataframe() # temp, salinity, pressure
df = df.dropna().reset_index()              # drop bad rows
df.to_csv("clean.csv", index=False)         # ready for Postgres
```

Say aloud: “xarray opens the NetCDF, I pull the variables I need, drop missing values, and write CSV for the Postgres load.”

The hooks to drop on purpose

Plant these and pause:

1. Natural-language-to-SQL with schema-aware prompting
2. Query validation (SELECT-only)
3. RAG + vector similarity search
4. ETL pipeline (NetCDF -> Postgres)
5. Mistral 7B over a frontier model, and why
6. Async FastAPI
7. “This is the same pattern as SAP’s Joule”

Every one has a full answer above. And always land it on the SAP bridge: natural language to real data is exactly what Joule does.